C Programming Laboratory

Source Code for All 120 Experiments

Course Outcomes CO1 to CO5

CO	Topics covered	Experiments	Count
CO1	Introduction	1 - 24	24
CO2	Operators, Decision Making, Looping	25 - 48	24
CO3	Arrays, Searching, Sorting, Merging, Two-Dimensional Arrays, Strings	49 - 72	24
CO4	Structures, Nested Structures, Unions	73 - 96	24
CO5	Functions, Recursion, Pointers, File Handling	97 - 120	24

How to compile and run

- Compile with GCC: gcc program.c -o program -lm (the -lm flag is needed only for programs that use math.h).
- Run with: ./program (Linux/macOS) or program.exe (Windows).
- Every program was compiled with gcc -Wall without errors or warnings and uses standard C only.
- Experiment 120 expects students.dat, which is created by Experiment 119, so run 119 first.

CO1: Introduction to C Programming

1. Hello World Program with Structured Layout

Topic: Introduction | Course Outcome: CO1

```c
/* Structured layout: documentation, preprocessor, main, return */
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

2. Display Size and Range of All Primitive Data Types using sizeof operator

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("char     : %zu byte(s), %d to %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
    printf("short    : %zu byte(s), %d to %d\n", sizeof(short), SHRT_MIN, SHRT_MAX);
    printf("int      : %zu byte(s), %d to %d\n", sizeof(int), INT_MIN, INT_MAX);
    printf("long     : %zu byte(s), %ld to %ld\n", sizeof(long), LONG_MIN, LONG_MAX);
    printf("long long: %zu byte(s), %lld to %lld\n", sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("unsigned int: %zu byte(s), 0 to %u\n", sizeof(unsigned int), UINT_MAX);
    printf("float    : %zu byte(s), %e to %e\n", sizeof(float), FLT_MIN, FLT_MAX);
    printf("double   : %zu byte(s), %e to %e\n", sizeof(double), DBL_MIN, DBL_MAX);
    printf("long double: %zu byte(s)\n", sizeof(long double));
    return 0;
}
```

3. Add Two Integers Read from the User

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>

int main(void)
{
    int a, b;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    printf("Sum = %d\n", a + b);
    return 0;
}
```

4. Swap Two Numbers Using a Temporary Variable

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>

int main(void)
{
    int a, b, temp;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
```

```
    temp = a;
    a = b;
    b = temp;
    printf("After swapping: a = %d, b = %d\n", a, b);
    return 0;
}
```

5. Swap Two Numbers Without a Temporary Variable

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int main(void)
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    /* Method 1: arithmetic */
    a = a + b;
    b = a - b;
    a = a - b;
    printf("Arithmetic swap: a = %d, b = %d\n", a, b);

    /* Method 2: XOR (swap back) */
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;
    printf("XOR swap       : a = %d, b = %d\n", a, b);
    return 0;
}
```

6. Area and Circumference of a Circle Using #define

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>
#define PI 3.14159

int main(void)
{
    float r;
    printf("Enter radius: ");
    scanf("%f", &r);
    printf("Area          = %.2f\n", PI * r * r);
    printf("Circumference = %.2f\n", 2 * PI * r);
    return 0;
}
```

7. Simple Interest Calculation

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int main(void)
{
    float p, r, t, si;
    printf("Enter principal, rate (%%) and time (years): ");
    scanf("%f %f %f", &p, &r, &t);
    si = (p * r * t) / 100;
```

```
    printf("Simple Interest = %.2f\n", si);
    printf("Total Amount    = %.2f\n", p + si);
    return 0;
}
```

8. Compound Interest Calculation

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    double p, r, t, amount;
    printf("Enter principal, rate (%%) and time (years): ");
    scanf("%lf %lf %lf", &p, &r, &t);
    amount = p * pow(1 + r / 100, t);
    printf("Compound Interest = %.2f\n", amount - p);
    printf("Total Amount      = %.2f\n", amount);
    return 0;
}
/* Compile: gcc prog.c -lm */
```

9. Temperature Conversion (Celsius to Fahrenheit and Back)

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int main(void)
{
    float c, f;
    printf("Enter temperature in Celsius: ");
    scanf("%f", &c);
    printf("%.2f C = %.2f F\n", c, c * 9 / 5 + 32);

    printf("Enter temperature in Fahrenheit: ");
    scanf("%f", &f);
    printf("%.2f F = %.2f C\n", f, (f - 32) * 5 / 9);
    return 0;
}
```

10. Convert Days into Years, Weeks and Days

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int main(void)
{
    int days, years, weeks, rem;
    printf("Enter number of days: ");
    scanf("%d", &days);
    years = days / 365;
    rem   = days % 365;
    weeks = rem / 7;
    rem   = rem % 7;
    printf("%d years, %d weeks, %d days\n", years, weeks, rem);
    return 0;
}
```

11. Print the ASCII Value of a Character

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>

int main(void)
{
    char ch;
    printf("Enter a character: ");
    scanf("%c", &ch);
    printf("ASCII value of '%c' = %d\n", ch, ch);
    return 0;
}
```

12. Implicit and Explicit Type Casting

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>

int main(void)
{
    int a = 7, b = 2;
    float implicitResult, explicitResult;
    char c = 'A';

    /* Implicit: int + char promoted to int; int assigned to float */
    int sum = a + c;
    implicitResult = a / b;               /* integer division, then converted */
    printf("Implicit: 'A' + 7 = %d, a/b = %.2f\n", sum, implicitResult);

    /* Explicit: cast operator */
    explicitResult = (float)a / b;
    printf("Explicit: (float)a/b = %.2f\n", explicitResult);
    printf("Explicit: (int)3.99 = %d\n", (int)3.99);
    printf("Explicit: (char)66 = %c\n", (char)66);
    return 0;
}
```

13. Constants Using const and #define

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>
#define MAX_STUDENTS 60
#define COLLEGE "ABC Institute"

int main(void)
{
    const float PI = 3.14159f;
    const int DAYS_IN_WEEK = 7;

    printf("College      : %s\n", COLLEGE);
    printf("Max students : %d\n", MAX_STUDENTS);
    printf("PI           : %.5f\n", PI);
    printf("Days in week : %d\n", DAYS_IN_WEEK);
    /* PI = 3.14;  -> error: assignment of read-only variable */
    return 0;
}
```

14. Macros with Arguments (Square, Maximum)

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main(void)
{
    int n = 5, x = 12, y = 30;
    printf("Square of %d = %d\n", n, SQUARE(n));
    printf("Square of %d = %d\n", n + 1, SQUARE(n + 1));
    printf("Max of %d and %d = %d\n", x, y, MAX(x, y));
    return 0;
}
```

15. Conditional Compilation Using #ifdef / #ifndef

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>
#define DEBUG          /* comment this line to disable debug output */

#ifndef VERSION
#define VERSION 1
#endif

int main(void)
{
#ifdef DEBUG
    printf("Debug mode is ON\n");
#else
    printf("Debug mode is OFF\n");
#endif

#if VERSION == 1
    printf("Running version 1\n");
#else
    printf("Running a later version\n");
#endif
    return 0;
}
```

16. Area of a Triangle Using Heron's Formula

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    double a, b, c, s, area;
    printf("Enter three sides: ");
    scanf("%lf %lf %lf", &a, &b, &c);
    if (a + b > c && b + c > a && a + c > b) {
        s = (a + b + c) / 2;
        area = sqrt(s * (s - a) * (s - b) * (s - c));
        printf("Area = %.2f\n", area);
    } else {
        printf("Invalid triangle\n");
    }
```

```
    return 0;
}
```

17. Total and Average Marks of Five Subjects

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int main(void)
{
    float m1, m2, m3, m4, m5, total, avg;
    printf("Enter marks of five subjects: ");
    scanf("%f %f %f %f %f", &m1, &m2, &m3, &m4, &m5);
    total = m1 + m2 + m3 + m4 + m5;
    avg = total / 5;
    printf("Total   = %.2f\n", total);
    printf("Average = %.2f\n", avg);
    return 0;
}
```

18. Convert Seconds to HH:MM:SS

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int main(void)
{
    long sec;
    int h, m, s;
    printf("Enter seconds: ");
    scanf("%ld", &sec);
    h = sec / 3600;
    m = (sec % 3600) / 60;
    s = sec % 60;
    printf("%02d:%02d:%02d\n", h, m, s);
    return 0;
}
```

19. Extract the Digits of a Three-Digit Number

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int main(void)
{
    int n, hundreds, tens, units;
    printf("Enter a three-digit number: ");
    scanf("%d", &n);
    hundreds = n / 100;
    tens     = (n / 10) % 10;
    units    = n % 10;
    printf("Hundreds = %d, Tens = %d, Units = %d\n", hundreds, tens, units);
    return 0;
}
```

20. Distance Between Two Points

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double x1, y1, x2, y2, d;
    printf("Enter x1 y1: ");
    scanf("%lf %lf", &x1, &y1);
    printf("Enter x2 y2: ");
    scanf("%lf %lf", &x2, &y2);
    d = sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
    printf("Distance = %.3f\n", d);
    return 0;
}
```

21. Formatted Output Using printf Width and Precision

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>

int main(void)
{
    int n = 42;
    double pi = 3.14159265;
    char name[] = "Claude";

    printf("[%d]\n", n);
    printf("[%6d]\n", n);          /* width 6, right aligned */
    printf("[%-6d]\n", n);         /* left aligned */
    printf("[%06d]\n", n);         /* zero padded */
    printf("[%f]\n", pi);
    printf("[%.2f]\n", pi);        /* precision 2 */
    printf("[%10.3f]\n", pi);      /* width 10, precision 3 */
    printf("[%-10.1f]\n", pi);
    printf("[%10s]\n", name);
    printf("[%.3s]\n", name);      /* only first 3 characters */
    printf("[%*d]\n", 8, n);       /* width taken from argument */
    return 0;
}
```

22. Integer Overflow and Underflow Demonstration

Topic: Introduction | Course Outcome: CO1

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    /* Unsigned arithmetic wraps around by definition */
    unsigned char uc = 255;
    uc = uc + 1;
    printf("unsigned char 255 + 1 = %u (overflow)\n", uc);

    unsigned char under = 0;
    under = under - 1;
    printf("unsigned char 0 - 1   = %u (underflow)\n", under);
```

```
    unsigned int ui = UINT_MAX;
    printf("UINT_MAX + 1 = %u\n", ui + 1);

    /* Signed overflow is undefined behaviour; show the safe check instead */
    int x = INT_MAX;
    if (x > INT_MAX - 1)
        printf("x + 1 would overflow int\n");

    short s = SHRT_MAX;
    s = (short)(s + 1);          /* implementation-defined: usually wraps */
    printf("short %d + 1 = %d\n", SHRT_MAX, s);
    return 0;
}
```

23. Days of the Week Using enum

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

enum Day { MON = 1, TUE, WED, THU, FRI, SAT, SUN };

int main(void)
{
    const char *names[] = {"", "Monday", "Tuesday", "Wednesday",
                           "Thursday", "Friday", "Saturday", "Sunday"};
    enum Day d;
    for (d = MON; d <= SUN; d++)
        printf("%d : %s\n", d, names[d]);
    return 0;
}
```

24. Storage Classes (auto, static, extern, register)

Topic: Introduction | Course Outcome: CO1

```
#include <stdio.h>

int globalVar = 100;            /* external linkage (extern) */

void counter(void)
{
    static int count = 0;       /* retains value between calls */
    int autoVar = 0;            /* re-initialised every call */
    count++;
    autoVar++;
    printf("static count = %d, auto var = %d\n", count, autoVar);
}

int main(void)
{
    extern int globalVar;       /* declaration of the global */
    register int i;             /* hint: keep in CPU register */
    auto int a = 10;            /* default storage class of locals */

    printf("auto a = %d, extern globalVar = %d\n", a, globalVar);
    for (i = 0; i < 3; i++)
        counter();
    return 0;
}
```

CO2: Operators, Decision Making and Looping

25. Arithmetic Operators Demonstration

Topic: Operators | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int a = 17, b = 5;
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a %% b = %d\n", a % b);
    printf("a / (float)b = %.2f\n", a / (float)b);
    printf("a++ = %d\n", a++);
    printf("a after a++ = %d\n", a);
    printf("++a = %d\n", ++a);
    return 0;
}
```

26. Relational and Logical Operators Demonstration

Topic: Operators | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int a = 10, b = 20;
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);
    printf("a >  b : %d\n", a > b);
    printf("a <  b : %d\n", a < b);
    printf("a >= b : %d\n", a >= b);
    printf("a <= b : %d\n", a <= b);
    printf("(a<b) && (b>15) : %d\n", (a < b) && (b > 15));
    printf("(a>b) || (b>15) : %d\n", (a > b) || (b > 15));
    printf("!(a<b)          : %d\n", !(a < b));
    return 0;
}
```

27. Bitwise Operators Demonstration

Topic: Operators | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    unsigned int a = 12, b = 10;     /* 1100 and 1010 */
    printf("a & b  = %u\n", a & b);
    printf("a | b  = %u\n", a | b);
    printf("a ^ b  = %u\n", a ^ b);
    printf("~a     = %u\n", ~a);
    printf("a << 2 = %u\n", a << 2);
    printf("a >> 2 = %u\n", a >> 2);
    return 0;
}
```

28. Even or Odd Using Modulus and Bitwise AND Operators

Topic: Operators | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("Modulus method : %d is even\n", n);
    else
        printf("Modulus method : %d is odd\n", n);

    if ((n & 1) == 0)
        printf("Bitwise method : %d is even\n", n);
    else
        printf("Bitwise method : %d is odd\n", n);
    return 0;
}
```

29. Largest of Three Numbers Using if-else

Topic: Decision Making | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int a, b, c;
    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);
    if (a >= b && a >= c)
        printf("%d is the largest\n", a);
    else if (b >= a && b >= c)
        printf("%d is the largest\n", b);
    else
        printf("%d is the largest\n", c);
    return 0;
}
```

30. Leap Year Check

Topic: Decision Making | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int year;
    printf("Enter a year: ");
    scanf("%d", &year);
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("%d is a leap year\n", year);
    else
        printf("%d is not a leap year\n", year);
    return 0;
}
```

31. Simple Calculator Using switch

Topic: Decision Making | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    double a, b;
    char op;
    printf("Enter expression (e.g. 5 + 3): ");
    scanf("%lf %c %lf", &a, &op, &b);
    switch (op) {
        case '+': printf("%.2f\n", a + b); break;
        case '-': printf("%.2f\n", a - b); break;
        case '*': printf("%.2f\n", a * b); break;
        case '/':
            if (b != 0) printf("%.2f\n", a / b);
            else        printf("Error: division by zero\n");
            break;
        default: printf("Invalid operator\n");
    }
    return 0;
}
```

32. Grade Assignment Using else-if Ladder

Topic: Decision Making | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    float marks;
    printf("Enter marks (0-100): ");
    scanf("%f", &marks);
    if (marks < 0 || marks > 100)
        printf("Invalid marks\n");
    else if (marks >= 90) printf("Grade: O\n");
    else if (marks >= 80) printf("Grade: A+\n");
    else if (marks >= 70) printf("Grade: A\n");
    else if (marks >= 60) printf("Grade: B+\n");
    else if (marks >= 50) printf("Grade: B\n");
    else if (marks >= 40) printf("Grade: C\n");
    else                  printf("Grade: F (Fail)\n");
    return 0;
}
```

33. Roots of a Quadratic Equation

Topic: Decision Making | Course Outcome: CO2

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    double a, b, c, d, r1, r2;
    printf("Enter a, b, c: ");
    scanf("%lf %lf %lf", &a, &b, &c);
    if (a == 0) {
        printf("Not a quadratic equation\n");
        return 0;
```

```
    }
    d = b * b - 4 * a * c;
    if (d > 0) {
        r1 = (-b + sqrt(d)) / (2 * a);
        r2 = (-b - sqrt(d)) / (2 * a);
        printf("Real and distinct roots: %.3f, %.3f\n", r1, r2);
    } else if (d == 0) {
        printf("Real and equal roots: %.3f\n", -b / (2 * a));
    } else {
        printf("Complex roots: %.3f + %.3fi and %.3f - %.3fi\n",
               -b / (2 * a), sqrt(-d) / (2 * a),
               -b / (2 * a), sqrt(-d) / (2 * a));
    }
    return 0;
}
```

34. Vowel or Consonant Using switch

Topic: Decision Making | Course Outcome: CO2

```
#include <stdio.h>
#include <ctype.h>

int main(void)
{
    char ch;
    printf("Enter an alphabet: ");
    scanf(" %c", &ch);
    if (!isalpha(ch)) {
        printf("Not an alphabet\n");
        return 0;
    }
    switch (tolower(ch)) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            printf("%c is a vowel\n", ch);
            break;
        default:
            printf("%c is a consonant\n", ch);
    }
    return 0;
}
```

35. Largest and Smallest Using the Ternary Operator

Topic: Decision Making | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int a, b, c, large, small;
    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);
    large = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
    small = (a < b) ? ((a < c) ? a : c) : ((b < c) ? b : c);
    printf("Largest  = %d\nSmallest = %d\n", large, small);
    return 0;
}
```

36. Sum of the First N Natural Numbers

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, i, sum = 0;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        sum += i;
    printf("Sum (loop)    = %d\n", sum);
    printf("Sum (formula) = %d\n", n * (n + 1) / 2);
    return 0;
}
```

37. Factorial of a Number Using Loops

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, i;
    unsigned long long fact = 1;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 0) {
        printf("Factorial is not defined for negative numbers\n");
        return 0;
    }
    for (i = 2; i <= n; i++)
        fact *= i;
    printf("%d! = %llu\n", n, fact);
    return 0;
}
```

38. Fibonacci Series

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, i;
    long long a = 0, b = 1, next;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("%lld ", a);
        next = a + b;
        a = b;
        b = next;
    }
    printf("\n");
    return 0;
}
```

39. Prime Number Check

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, i, isPrime = 1;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 2)
        isPrime = 0;
    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            isPrime = 0;
            break;
        }
    }
    printf("%d is %s\n", n, isPrime ? "prime" : "not prime");
    return 0;
}
```

40. Prime Numbers in a Given Range

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int lo, hi, n, i, isPrime;
    printf("Enter range (low high): ");
    scanf("%d %d", &lo, &hi);
    printf("Primes between %d and %d:\n", lo, hi);
    for (n = lo; n <= hi; n++) {
        if (n < 2) continue;
        isPrime = 1;
        for (i = 2; i * i <= n; i++)
            if (n % i == 0) { isPrime = 0; break; }
        if (isPrime)
            printf("%d ", n);
    }
    printf("\n");
    return 0;
}
```

41. Armstrong Number

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    int n, temp, digits = 0, rem;
    double sum = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (temp = n; temp > 0; temp /= 10)
        digits++;
    for (temp = n; temp > 0; temp /= 10) {
```

```
        rem = temp % 10;
        sum += pow(rem, digits);
    }
    if ((int)(sum + 0.5) == n)
        printf("%d is an Armstrong number\n", n);
    else
        printf("%d is not an Armstrong number\n", n);
    return 0;
}
```

42. Palindrome Number

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, temp, rev = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (temp = n; temp > 0; temp /= 10)
        rev = rev * 10 + temp % 10;
    if (rev == n)
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome\n", n);
    return 0;
}
```

43. Reverse a Number

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, rev = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    while (n != 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    printf("Reversed number = %d\n", rev);
    return 0;
}
```

44. GCD and LCM of Two Numbers

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int a, b, x, y, t;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    x = a; y = b;
    while (y != 0) {          /* Euclid's algorithm */
```

```
        t = y;
        y = x % y;
        x = t;
    }
    printf("GCD = %d\n", x);
    printf("LCM = %d\n", (a / x) * b);
    return 0;
}
```

45. Perfect Number Check

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, i, sum = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (i = 1; i <= n / 2; i++)
        if (n % i == 0)
            sum += i;
    if (sum == n && n > 0)
        printf("%d is a perfect number\n", n);
    else
        printf("%d is not a perfect number\n", n);
    return 0;
}
```

46. Star Pyramid and Diamond Patterns

Topic: Looping | Course Outcome: CO2

```
#include <stdio.h>

int main(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    printf("\nPyramid:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }

    printf("\nDiamond:\n");
    for (i = 1; i <= n; i++) {                /* upper half */
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }
    for (i = n - 1; i >= 1; i--) {            /* lower half */
        for (j = 1; j <= n - i; j++) printf(" ");
        for (j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }
    return 0;
}
```

47. Floyd's Triangle and Pascal's Triangle

Topic: Looping | Course Outcome: CO2

```c
#include <stdio.h>

int main(void)
{
    int n, i, j, num = 1, coef;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    printf("\nFloyd's Triangle:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++)
            printf("%d ", num++);
        printf("\n");
    }

    printf("\nPascal's Triangle:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n - i - 1; j++) printf("  ");
        coef = 1;
        for (j = 0; j <= i; j++) {
            printf("%4d", coef);
            coef = coef * (i - j) / (j + 1);
        }
        printf("\n");
    }
    return 0;
}
```

48. Menu-Driven Program Using do-while, break and continue

Topic: Looping | Course Outcome: CO2

```c
#include <stdio.h>

int main(void)
{
    int choice, a, b;
    do {
        printf("\n--- MENU ---\n1. Add\n2. Subtract\n3. Multiply\n4. Divide\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        if (choice == 5)
            break;                          /* leave the loop */
        if (choice < 1 || choice > 4) {
            printf("Invalid choice, try again.\n");
            continue;                       /* skip to next iteration */
        }
        printf("Enter two numbers: ");
        scanf("%d %d", &a, &b);
        switch (choice) {
            case 1: printf("Result = %d\n", a + b); break;
            case 2: printf("Result = %d\n", a - b); break;
            case 3: printf("Result = %d\n", a * b); break;
            case 4:
                if (b == 0) printf("Cannot divide by zero\n");
                else        printf("Result = %.2f\n", (float)a / b);
                break;
```

```
        }
    } while (1);
    printf("Program terminated.\n");
    return 0;
}
```

CO3: Arrays, Searching, Sorting, Matrices and Strings

49. Read and Display a One-Dimensional Array

Topic: Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Array elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

50. Find the Largest and Smallest Element in an Array

Topic: Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, max, min;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    max = min = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] > max) max = a[i];
        if (a[i] < min) min = a[i];
    }
    printf("Largest = %d, Smallest = %d\n", max, min);
    return 0;
}
```

51. Reverse an Array

Topic: Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, temp;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n / 2; i++) {
        temp = a[i];
        a[i] = a[n - 1 - i];
```

```
        a[n - 1 - i] = temp;
    }
    printf("Reversed array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

52. Linear Search

Topic: Searching | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, key, found = -1;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter element to search: ");
    scanf("%d", &key);
    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            found = i;
            break;
        }
    }
    if (found >= 0)
        printf("%d found at position %d (index %d)\n", key, found + 1, found);
    else
        printf("%d not found\n", key);
    return 0;
}
```

53. Binary Search

Topic: Searching | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, key, low, high, mid, found = -1;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements in ascending order: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter element to search: ");
    scanf("%d", &key);
    low = 0;
    high = n - 1;
    while (low <= high) {
        mid = low + (high - low) / 2;
        if (a[mid] == key) { found = mid; break; }
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
```

```
    if (found >= 0)
        printf("%d found at index %d\n", key, found);
    else
        printf("%d not found\n", key);
    return 0;
}
```

54. Bubble Sort

Topic: Sorting | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, j, temp, swapped;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

55. Selection Sort

Topic: Sorting | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, j, minIdx, temp;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++) {
        minIdx = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[minIdx])
                minIdx = j;
        temp = a[i]; a[i] = a[minIdx]; a[minIdx] = temp;
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
```

```
    printf("\n");
    return 0;
}
```

56. Insertion Sort

Topic: Sorting | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, j, key;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

57. Merge Two Sorted Arrays

Topic: Merging | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[50], b[50], c[100], n1, n2, i = 0, j = 0, k = 0;
    printf("Size of first sorted array: ");
    scanf("%d", &n1);
    printf("Enter elements: ");
    for (i = 0; i < n1; i++) scanf("%d", &a[i]);
    printf("Size of second sorted array: ");
    scanf("%d", &n2);
    printf("Enter elements: ");
    for (j = 0; j < n2; j++) scanf("%d", &b[j]);

    i = j = 0;
    while (i < n1 && j < n2)
        c[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < n1) c[k++] = a[i++];
    while (j < n2) c[k++] = b[j++];

    printf("Merged array: ");
    for (i = 0; i < k; i++)
        printf("%d ", c[i]);
    printf("\n");
```

```
    return 0;
}
```

58. Remove Duplicate Elements from an Array

Topic: Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[100], n, i, j, k;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; ) {
            if (a[j] == a[i]) {
                for (k = j; k < n - 1; k++)
                    a[k] = a[k + 1];
                n--;
            } else {
                j++;
            }
        }
    }
    printf("Array without duplicates: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

59. Matrix Addition and Subtraction

Topic: Two-Dimensional Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[10][10], b[10][10], r, c, i, j;
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);
    printf("Enter matrix A:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
    printf("Enter matrix B:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &b[i][j]);

    printf("A + B:\n");
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%4d", a[i][j] + b[i][j]);
        printf("\n");
    }
    printf("A - B:\n");
```

```
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%4d", a[i][j] - b[i][j]);
        printf("\n");
    }
    return 0;
}
```

60. Matrix Multiplication

Topic: Two-Dimensional Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[10][10], b[10][10], c[10][10] = {0};
    int r1, c1, r2, c2, i, j, k;
    printf("Enter rows and columns of A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of B: ");
    scanf("%d %d", &r2, &c2);
    if (c1 != r2) {
        printf("Multiplication not possible (cols of A != rows of B)\n");
        return 0;
    }
    printf("Enter matrix A:\n");
    for (i = 0; i < r1; i++)
        for (j = 0; j < c1; j++)
            scanf("%d", &a[i][j]);
    printf("Enter matrix B:\n");
    for (i = 0; i < r2; i++)
        for (j = 0; j < c2; j++)
            scanf("%d", &b[i][j]);

    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++)
            for (k = 0; k < c1; k++)
                c[i][j] += a[i][k] * b[k][j];

    printf("A x B:\n");
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++)
            printf("%5d", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

61. Transpose of a Matrix

Topic: Two-Dimensional Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[10][10], t[10][10], r, c, i, j;
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);
    printf("Enter matrix:\n");
    for (i = 0; i < r; i++)
```

```
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            t[j][i] = a[i][j];
    printf("Transpose:\n");
    for (i = 0; i < c; i++) {
        for (j = 0; j < r; j++)
            printf("%4d", t[i][j]);
        printf("\n");
    }
    return 0;
}
```

62. Row-wise and Column-wise Sum of a Matrix

Topic: Two-Dimensional Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[10][10], r, c, i, j, sum;
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);
    printf("Enter matrix:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
    for (i = 0; i < r; i++) {
        sum = 0;
        for (j = 0; j < c; j++)
            sum += a[i][j];
        printf("Sum of row %d = %d\n", i + 1, sum);
    }
    for (j = 0; j < c; j++) {
        sum = 0;
        for (i = 0; i < r; i++)
            sum += a[i][j];
        printf("Sum of column %d = %d\n", j + 1, sum);
    }
    return 0;
}
```

63. Diagonal Sums and Symmetric Matrix Check

Topic: Two-Dimensional Arrays | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    int a[10][10], n, i, j, principal = 0, secondary = 0, symmetric = 1;
    printf("Enter order of square matrix: ");
    scanf("%d", &n);
    printf("Enter matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    for (i = 0; i < n; i++) {
        principal += a[i][i];
        secondary += a[i][n - 1 - i];
```

```
    }
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (a[i][j] != a[j][i])
                symmetric = 0;
    printf("Principal diagonal sum = %d\n", principal);
    printf("Secondary diagonal sum = %d\n", secondary);
    printf("The matrix is %s\n", symmetric ? "symmetric" : "not symmetric");
    return 0;
}
```

64. String Length Without strlen

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    char str[100];
    int len = 0;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    while (str[len] != '\0' && str[len] != '\n')
        len++;
    printf("Length = %d\n", len);
    return 0;
}
```

65. Reverse a String

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    char str[100], temp;
    int len = 0, i;
    printf("Enter a string: ");
    scanf("%99[^\n]", str);
    while (str[len] != '\0') len++;
    for (i = 0; i < len / 2; i++) {
        temp = str[i];
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = temp;
    }
    printf("Reversed string: %s\n", str);
    return 0;
}
```

66. Palindrome String Check

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    char str[100];
    int len = 0, i, isPal = 1;
    printf("Enter a string: ");
```

```
    scanf("%99s", str);
    while (str[len] != '\0') len++;
    for (i = 0; i < len / 2; i++) {
        if (str[i] != str[len - 1 - i]) {
            isPal = 0;
            break;
        }
    }
    printf("\"%s\" is %s\n", str, isPal ? "a palindrome" : "not a palindrome");
    return 0;
}
```

67. Count Vowels, Consonants, Digits and Spaces

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>
#include <ctype.h>

int main(void)
{
    char str[200];
    int i, v = 0, c = 0, d = 0, s = 0;
    char ch;
    printf("Enter a line of text: ");
    fgets(str, sizeof(str), stdin);
    for (i = 0; str[i] != '\0'; i++) {
        ch = tolower(str[i]);
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
            v++;
        else if (ch >= 'a' && ch <= 'z')
            c++;
        else if (ch >= '0' && ch <= '9')
            d++;
        else if (ch == ' ')
            s++;
    }
    printf("Vowels = %d, Consonants = %d, Digits = %d, Spaces = %d\n", v, c, d, s);
    return 0;
}
```

68. String Concatenation and Comparison Without Library Functions

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    char s1[100], s2[100];
    int i, j, diff;
    printf("Enter first string : ");
    scanf("%49s", s1);
    printf("Enter second string: ");
    scanf("%49s", s2);

    /* Comparison */
    i = 0;
    while (s1[i] != '\0' && s1[i] == s2[i])
        i++;
    diff = s1[i] - s2[i];
    if (diff == 0)      printf("Strings are equal\n");
```

```
    else if (diff < 0)  printf("First string is smaller\n");
    else                printf("First string is greater\n");

    /* Concatenation: append s2 to s1 */
    for (i = 0; s1[i] != '\0'; i++)
        ;
    for (j = 0; s2[j] != '\0'; j++, i++)
        s1[i] = s2[j];
    s1[i] = '\0';
    printf("Concatenated string: %s\n", s1);
    return 0;
}
```

69. Anagram Check

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>
#include <ctype.h>

int main(void)
{
    char s1[100], s2[100];
    int count[26] = {0}, i, anagram = 1;
    printf("Enter first string : ");
    scanf("%99s", s1);
    printf("Enter second string: ");
    scanf("%99s", s2);
    for (i = 0; s1[i] != '\0'; i++)
        if (isalpha(s1[i])) count[tolower(s1[i]) - 'a']++;
    for (i = 0; s2[i] != '\0'; i++)
        if (isalpha(s2[i])) count[tolower(s2[i]) - 'a']--;
    for (i = 0; i < 26; i++)
        if (count[i] != 0) { anagram = 0; break; }
    printf("The strings are %s\n", anagram ? "anagrams" : "not anagrams");
    return 0;
}
```

70. Sort Strings in Alphabetical Order

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char names[20][50], temp[50];
    int n, i, j;
    printf("Enter number of strings: ");
    scanf("%d", &n);
    printf("Enter %d strings:\n", n);
    for (i = 0; i < n; i++)
        scanf("%49s", names[i]);
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (strcmp(names[i], names[j]) > 0) {
                strcpy(temp, names[i]);
                strcpy(names[i], names[j]);
                strcpy(names[j], temp);
            }
        }
```

```
    }
    printf("Sorted order:\n");
    for (i = 0; i < n; i++)
        printf("%s\n", names[i]);
    return 0;
}
```

71. Character Frequency in a String

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    char str[200];
    int freq[256] = {0}, i;
    printf("Enter a string: ");
    scanf("%199[^\n]", str);
    for (i = 0; str[i] != '\0'; i++)
        freq[(unsigned char)str[i]]++;
    printf("Character frequencies:\n");
    for (i = 0; i < 256; i++)
        if (freq[i] > 0)
            printf("'%c' : %d\n", i, freq[i]);
    return 0;
}
```

72. Substring (Pattern) Search in a String

Topic: Strings | Course Outcome: CO3

```
#include <stdio.h>

int main(void)
{
    char text[200], pat[50];
    int i, j, found = 0;
    printf("Enter the text   : ");
    scanf("%199[^\n]", text);
    printf("Enter the pattern: ");
    scanf(" %49[^\n]", pat);
    for (i = 0; text[i] != '\0'; i++) {
        for (j = 0; pat[j] != '\0' && text[i + j] == pat[j]; j++)
            ;
        if (pat[j] == '\0') {
            printf("Pattern found at index %d\n", i);
            found = 1;
        }
    }
    if (!found)
        printf("Pattern not found\n");
    return 0;
}
```

CO4: Structures and Unions

73. Student Record Using a Structure

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

int main(void)
{
    struct Student s;
    printf("Enter roll number: ");
    scanf("%d", &s.roll);
    printf("Enter name: ");
    scanf(" %49[^\n]", s.name);
    printf("Enter marks: ");
    scanf("%f", &s.marks);
    printf("\n--- Student Record ---\n");
    printf("Roll : %d\nName : %s\nMarks: %.2f\n", s.roll, s.name, s.marks);
    return 0;
}
```

74. Employee Details Using an Array of Structures

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Employee {
    int id;
    char name[50];
    float salary;
};

int main(void)
{
    struct Employee emp[50];
    int n, i;
    printf("Enter number of employees: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Employee %d - id, name, salary: ", i + 1);
        scanf("%d %49s %f", &emp[i].id, emp[i].name, &emp[i].salary);
    }
    printf("\n%-5s %-20s %10s\n", "ID", "Name", "Salary");
    for (i = 0; i < n; i++)
        printf("%-5d %-20s %10.2f\n", emp[i].id, emp[i].name, emp[i].salary);
    return 0;
}
```

75. Complex Number Addition and Multiplication

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Complex {
```

```
    float real, imag;
};

struct Complex add(struct Complex a, struct Complex b)
{
    struct Complex r = { a.real + b.real, a.imag + b.imag };
    return r;
}

struct Complex multiply(struct Complex a, struct Complex b)
{
    struct Complex r;
    r.real = a.real * b.real - a.imag * b.imag;
    r.imag = a.real * b.imag + a.imag * b.real;
    return r;
}

int main(void)
{
    struct Complex c1, c2, sum, prod;
    printf("Enter first complex number (real imag): ");
    scanf("%f %f", &c1.real, &c1.imag);
    printf("Enter second complex number (real imag): ");
    scanf("%f %f", &c2.real, &c2.imag);
    sum = add(c1, c2);
    prod = multiply(c1, c2);
    printf("Sum     = %.2f + %.2fi\n", sum.real, sum.imag);
    printf("Product = %.2f + %.2fi\n", prod.real, prod.imag);
    return 0;
}
```

76. Distance Between Two Points Using Structures

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>
#include <math.h>

struct Point {
    double x, y;
};

int main(void)
{
    struct Point p1, p2;
    double d;
    printf("Enter coordinates of point 1 (x y): ");
    scanf("%lf %lf", &p1.x, &p1.y);
    printf("Enter coordinates of point 2 (x y): ");
    scanf("%lf %lf", &p2.x, &p2.y);
    d = sqrt((p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) * (p2.y - p1.y));
    printf("Distance = %.3f\n", d);
    return 0;
}
```

77. Time Difference Using Structures

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Time {
```

```
    int hours, minutes, seconds;
};

int main(void)
{
    struct Time t1, t2, diff;
    long s1, s2, d;
    printf("Enter start time (hh mm ss): ");
    scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds);
    printf("Enter end time   (hh mm ss): ");
    scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds);

    s1 = t1.hours * 3600L + t1.minutes * 60 + t1.seconds;
    s2 = t2.hours * 3600L + t2.minutes * 60 + t2.seconds;
    d = s2 - s1;
    if (d < 0) d = -d;

    diff.hours = d / 3600;
    diff.minutes = (d % 3600) / 60;
    diff.seconds = d % 60;
    printf("Time difference = %02d:%02d:%02d\n", diff.hours, diff.minutes, diff.seconds);
    return 0;
}
```

78. Date Validation and Comparison

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Date {
    int day, month, year;
};

int isLeap(int y)
{
    return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
}

int isValid(struct Date d)
{
    int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (d.year < 1 || d.month < 1 || d.month > 12 || d.day < 1)
        return 0;
    if (d.month == 2 && isLeap(d.year))
        return d.day <= 29;
    return d.day <= days[d.month - 1];
}

int compare(struct Date a, struct Date b)
{
    if (a.year != b.year)   return a.year < b.year ? -1 : 1;
    if (a.month != b.month) return a.month < b.month ? -1 : 1;
    if (a.day != b.day)     return a.day < b.day ? -1 : 1;
    return 0;
}

int main(void)
{
    struct Date d1, d2;
    int r;
```

```
    printf("Enter first date (dd mm yyyy): ");
    scanf("%d %d %d", &d1.day, &d1.month, &d1.year);
    printf("Enter second date (dd mm yyyy): ");
    scanf("%d %d %d", &d2.day, &d2.month, &d2.year);
    if (!isValid(d1) || !isValid(d2)) {
        printf("One or both dates are invalid\n");
        return 0;
    }
    r = compare(d1, d2);
    if (r == 0)      printf("Both dates are equal\n");
    else if (r < 0)  printf("First date is earlier\n");
    else             printf("First date is later\n");
    return 0;
}
```

79. Sort Students by Marks

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void)
{
    struct Student s[50], temp;
    int n, i, j;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Student %d - roll, name, marks: ", i + 1);
        scanf("%d %29s %f", &s[i].roll, s[i].name, &s[i].marks);
    }
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (s[j].marks < s[j + 1].marks) {      /* descending order */
                temp = s[j]; s[j] = s[j + 1]; s[j + 1] = temp;
            }
    printf("\nRank list (highest marks first):\n");
    for (i = 0; i < n; i++)
        printf("%d. %-5d %-15s %.2f\n", i + 1, s[i].roll, s[i].name, s[i].marks);
    return 0;
}
```

80. Search a Record by Roll Number

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void)
{
```

```
    struct Student s[50];
    int n, i, key, found = 0;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Student %d - roll, name, marks: ", i + 1);
        scanf("%d %29s %f", &s[i].roll, s[i].name, &s[i].marks);
    }
    printf("Enter roll number to search: ");
    scanf("%d", &key);
    for (i = 0; i < n; i++) {
        if (s[i].roll == key) {
            printf("Found: Roll %d, Name %s, Marks %.2f\n", s[i].roll, s[i].name, s[i].marks);
            found = 1;
            break;
        }
    }
    if (!found)
        printf("Record not found\n");
    return 0;
}
```

81. Employee with Address and Date of Birth (Nested Structure)

Topic: Nested Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Date {
    int day, month, year;
};

struct Address {
    char street[40];
    char city[30];
    int pin;
};

struct Employee {
    int id;
    char name[40];
    struct Date dob;
    struct Address addr;
};

int main(void)
{
    struct Employee e;
    printf("Enter id and name: ");
    scanf("%d %39s", &e.id, e.name);
    printf("Enter date of birth (dd mm yyyy): ");
    scanf("%d %d %d", &e.dob.day, &e.dob.month, &e.dob.year);
    printf("Enter street: ");
    scanf(" %39[^\n]", e.addr.street);
    printf("Enter city and PIN: ");
    scanf("%29s %d", e.addr.city, &e.addr.pin);

    printf("\nID      : %d\nName    : %s\n", e.id, e.name);
    printf("DOB     : %02d/%02d/%d\n", e.dob.day, e.dob.month, e.dob.year);
    printf("Address : %s, %s - %d\n", e.addr.street, e.addr.city, e.addr.pin);
    return 0;
```

```
}
```

82. Student Report Card with Nested Subject Marks

Topic: Nested Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Marks {
    float maths, physics, chemistry, english, cs;
};

struct Student {
    int roll;
    char name[40];
    struct Marks m;
};

int main(void)
{
    struct Student s;
    float total, avg;
    printf("Enter roll and name: ");
    scanf("%d %39s", &s.roll, s.name);
    printf("Enter marks (Maths Physics Chemistry English CS): ");
    scanf("%f %f %f %f %f", &s.m.maths, &s.m.physics, &s.m.chemistry, &s.m.english, &s.m.cs);
    total = s.m.maths + s.m.physics + s.m.chemistry + s.m.english + s.m.cs;
    avg = total / 5;

    printf("\n========== REPORT CARD ==========\n");
    printf("Roll: %d   Name: %s\n", s.roll, s.name);
    printf("Maths     : %.1f\nPhysics   : %.1f\nChemistry : %.1f\n", s.m.maths, s.m.physics,
s.m.chemistry);
    printf("English   : %.1f\nComputer  : %.1f\n", s.m.english, s.m.cs);
    printf("Total     : %.1f\nAverage   : %.2f\n", total, avg);
    printf("Result    : %s\n", avg >= 40 ? "PASS" : "FAIL");
    return 0;
}
```

83. Library Management Using Structures

Topic: Nested Structures | Course Outcome: CO4

```
#include <stdio.h>
#include <string.h>

struct Date {
    int day, month, year;
};

struct Book {
    int id;
    char title[50];
    char author[40];
    int issued;              /* 0 = available, 1 = issued */
    struct Date issueDate;
};

struct Book lib[100];
int count = 0;

void addBook(void)
```

```
{
    struct Book *b = &lib[count];
    printf("Enter book id, title, author: ");
    scanf("%d %49s %39s", &b->id, b->title, b->author);
    b->issued = 0;
    count++;
}

void display(void)
{
    int i;
    printf("\n%-5s %-20s %-15s %s\n", "ID", "Title", "Author", "Status");
    for (i = 0; i < count; i++)
        printf("%-5d %-20s %-15s %s\n", lib[i].id, lib[i].title, lib[i].author,
               lib[i].issued ? "Issued" : "Available");
}

void issueBook(void)
{
    int id, i;
    printf("Enter book id to issue: ");
    scanf("%d", &id);
    for (i = 0; i < count; i++) {
        if (lib[i].id == id) {
            if (lib[i].issued) { printf("Already issued\n"); return; }
            printf("Enter issue date (dd mm yyyy): ");
            scanf("%d %d %d", &lib[i].issueDate.day, &lib[i].issueDate.month,
&lib[i].issueDate.year);
            lib[i].issued = 1;
            printf("Book issued\n");
            return;
        }
    }
    printf("Book not found\n");
}

void returnBook(void)
{
    int id, i;
    printf("Enter book id to return: ");
    scanf("%d", &id);
    for (i = 0; i < count; i++)
        if (lib[i].id == id && lib[i].issued) {
            lib[i].issued = 0;
            printf("Book returned\n");
            return;
        }
    printf("Invalid return request\n");
}

int main(void)
{
    int ch;
    do {
        printf("\n1.Add 2.Display 3.Issue 4.Return 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: addBook(); break;
            case 2: display(); break;
            case 3: issueBook(); break;
            case 4: returnBook(); break;
```

```
        }
    } while (ch != 5);
    return 0;
}
```

84. Bank Account Management Using Structures

Topic: Nested Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Account {
    int accNo;
    char name[40];
    double balance;
};

struct Account acc[50];
int n = 0;

int find(int no)
{
    int i;
    for (i = 0; i < n; i++)
        if (acc[i].accNo == no)
            return i;
    return -1;
}

int main(void)
{
    int ch, no, idx;
    double amt;
    do {
        printf("\n1.Create 2.Deposit 3.Withdraw 4.Balance 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("Enter account no, name, opening balance: ");
                scanf("%d %39s %lf", &acc[n].accNo, acc[n].name, &acc[n].balance);
                n++;
                break;
            case 2:
                printf("Account no and amount: ");
                scanf("%d %lf", &no, &amt);
                idx = find(no);
                if (idx < 0) printf("Account not found\n");
                else { acc[idx].balance += amt; printf("New balance: %.2f\n",
acc[idx].balance); }
                break;
            case 3:
                printf("Account no and amount: ");
                scanf("%d %lf", &no, &amt);
                idx = find(no);
                if (idx < 0) printf("Account not found\n");
                else if (amt > acc[idx].balance) printf("Insufficient balance\n");
                else { acc[idx].balance -= amt; printf("New balance: %.2f\n",
acc[idx].balance); }
                break;
            case 4:
                printf("Account no: ");
```

```
                scanf("%d", &no);
                idx = find(no);
                if (idx < 0) printf("Account not found\n");
                else printf("%s's balance: %.2f\n", acc[idx].name, acc[idx].balance);
                break;
        }
    } while (ch != 5);
    return 0;
}
```

85. Employee Payroll Processing

Topic: Nested Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Salary {
    float basic, hra, da, pf, gross, net;
};

struct Employee {
    int id;
    char name[40];
    struct Salary sal;
};

int main(void)
{
    struct Employee e[50];
    int n, i;
    printf("Enter number of employees: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Employee %d - id, name, basic salary: ", i + 1);
        scanf("%d %39s %f", &e[i].id, e[i].name, &e[i].sal.basic);
        e[i].sal.hra   = 0.20f * e[i].sal.basic;     /* 20% HRA */
        e[i].sal.da    = 0.50f * e[i].sal.basic;     /* 50% DA  */
        e[i].sal.pf    = 0.12f * e[i].sal.basic;     /* 12% PF  */
        e[i].sal.gross = e[i].sal.basic + e[i].sal.hra + e[i].sal.da;
        e[i].sal.net   = e[i].sal.gross - e[i].sal.pf;
    }
    printf("\n%-5s %-15s %9s %9s %9s %9s %10s %10s\n",
           "ID", "Name", "Basic", "HRA", "DA", "PF", "Gross", "Net");
    for (i = 0; i < n; i++)
        printf("%-5d %-15s %9.2f %9.2f %9.2f %9.2f %10.2f %10.2f\n",
               e[i].id, e[i].name, e[i].sal.basic, e[i].sal.hra, e[i].sal.da,
               e[i].sal.pf, e[i].sal.gross, e[i].sal.net);
    return 0;
}
```

86. Inventory Management System

Topic: Nested Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Supplier {
    char name[30];
    char phone[15];
};

struct Item {
```

```
    int code;
    char name[30];
    int qty;
    float price;
    struct Supplier sup;
};

struct Item inv[100];
int n = 0;

int find(int code)
{
    int i;
    for (i = 0; i < n; i++)
        if (inv[i].code == code)
            return i;
    return -1;
}

int main(void)
{
    int ch, code, q, idx, i;
    float total;
    do {
        printf("\n1.Add item 2.Stock in 3.Stock out 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("Code, name, qty, price: ");
                scanf("%d %29s %d %f", &inv[n].code, inv[n].name, &inv[n].qty, &inv[n].price);
                printf("Supplier name and phone: ");
                scanf("%29s %14s", inv[n].sup.name, inv[n].sup.phone);
                n++;
                break;
            case 2:
                printf("Item code and quantity received: ");
                scanf("%d %d", &code, &q);
                idx = find(code);
                if (idx < 0) printf("Item not found\n"); else inv[idx].qty += q;
                break;
            case 3:
                printf("Item code and quantity sold: ");
                scanf("%d %d", &code, &q);
                idx = find(code);
                if (idx < 0) printf("Item not found\n");
                else if (q > inv[idx].qty) printf("Insufficient stock\n");
                else inv[idx].qty -= q;
                break;
            case 4:
                total = 0;
                printf("\n%-6s %-15s %5s %8s %-12s\n", "Code", "Name", "Qty", "Price",
"Supplier");
                for (i = 0; i < n; i++) {
                    printf("%-6d %-15s %5d %8.2f %-12s\n", inv[i].code, inv[i].name,
                           inv[i].qty, inv[i].price, inv[i].sup.name);
                    total += inv[i].qty * inv[i].price;
                }
                printf("Total stock value = %.2f\n", total);
                break;
        }
    } while (ch != 5);
```

```
    return 0;
}
```

87. Pass a Structure to a Function by Value

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Rectangle {
    float length, width;
};

float area(struct Rectangle r)          /* r is a copy of the caller's structure */
{
    float a = r.length * r.width;
    r.length = 0;                       /* changes only the local copy */
    return a;
}

int main(void)
{
    struct Rectangle rect;
    printf("Enter length and width: ");
    scanf("%f %f", &rect.length, &rect.width);
    printf("Area = %.2f\n", area(rect));
    printf("Length after the call (unchanged) = %.2f\n", rect.length);
    return 0;
}
```

88. Pass a Structure to a Function by Reference

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Rectangle {
    float length, width;
};

void scale(struct Rectangle *r, float factor)
{
    r->length *= factor;
    r->width  *= factor;
}

int main(void)
{
    struct Rectangle rect;
    printf("Enter length and width: ");
    scanf("%f %f", &rect.length, &rect.width);
    scale(&rect, 2);
    printf("After scaling by 2: length = %.2f, width = %.2f\n", rect.length, rect.width);
    return 0;
}
```

89. Pointer to a Structure and the Arrow Operator

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void)
{
    struct Student s = {101, "Arun", 87.5f};
    struct Student *p = &s;

    printf("Using dot operator  : %d %s %.1f\n", s.roll, s.name, s.marks);
    printf("Using arrow operator: %d %s %.1f\n", p->roll, p->name, p->marks);
    printf("Using (*p).member   : %d\n", (*p).roll);

    p->marks = 92.0f;
    printf("Updated marks through pointer = %.1f\n", s.marks);
    return 0;
}
```

90. Union Demonstration

Topic: Unions | Course Outcome: CO4

```
#include <stdio.h>
#include <string.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main(void)
{
    union Data d;
    d.i = 10;
    printf("d.i = %d\n", d.i);
    d.f = 220.5f;
    printf("d.f = %.1f (d.i is now garbage: %d)\n", d.f, d.i);
    strcpy(d.str, "Union");
    printf("d.str = %s (d.f is now garbage)\n", d.str);
    return 0;
}
```

91. Memory Size Comparison of Structure and Union

Topic: Unions | Course Outcome: CO4

```
#include <stdio.h>

struct S {
    int i;
    float f;
    char c[20];
};
```

```
union U {
    int i;
    float f;
    char c[20];
};

int main(void)
{
    printf("Size of structure = %zu bytes (sum of members + padding)\n", sizeof(struct S));
    printf("Size of union     = %zu bytes (largest member)\n", sizeof(union U));
    return 0;
}
```

92. Variant Data Type Using a Union

Topic: Unions | Course Outcome: CO4

```
#include <stdio.h>

enum Type { INT, FLOAT, CHAR };

struct Variant {
    enum Type type;
    union {
        int   i;
        float f;
        char  c;
    } value;
};

void print(struct Variant v)
{
    switch (v.type) {
        case INT:   printf("int   : %d\n", v.value.i); break;
        case FLOAT: printf("float : %.2f\n", v.value.f); break;
        case CHAR:  printf("char  : %c\n", v.value.c); break;
    }
}

int main(void)
{
    struct Variant a, b, c;
    a.type = INT;   a.value.i = 42;
    b.type = FLOAT; b.value.f = 3.14f;
    c.type = CHAR;  c.value.c = 'Z';
    print(a);
    print(b);
    print(c);
    return 0;
}
```

93. Structure with Bit Fields

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Date {
    unsigned int day   : 5;     /* 0-31  */
    unsigned int month : 4;     /* 0-15  */
    unsigned int year  : 12;    /* 0-4095 */
};
```

```
struct Flags {
    unsigned int isAdmin  : 1;
    unsigned int isActive : 1;
    unsigned int level    : 3;
};

int main(void)
{
    struct Date d = {9, 11, 2026};
    struct Flags f = {1, 1, 5};
    printf("Date  : %u/%u/%u\n", d.day, d.month, d.year);
    printf("Flags : admin=%u active=%u level=%u\n", f.isAdmin, f.isActive, f.level);
    printf("Size of struct Date  = %zu bytes\n", sizeof(struct Date));
    printf("Size of struct Flags = %zu bytes\n", sizeof(struct Flags));
    return 0;
}
```

94. typedef with Structures and Unions

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

typedef struct {
    int x, y;
} Point;

typedef union {
    int i;
    float f;
} Number;

typedef unsigned long ulong;

int main(void)
{
    Point p = {3, 4};
    Number n;
    ulong big = 4000000000UL;

    n.f = 2.5f;
    printf("Point  = (%d, %d)\n", p.x, p.y);
    printf("Number = %.1f\n", n.f);
    printf("ulong  = %lu\n", big);
    return 0;
}
```

95. Railway Ticket Reservation System

Topic: Nested Structures | Course Outcome: CO4

```
#include <stdio.h>
#include <string.h>

#define MAX_SEATS 10

struct Date {
    int day, month, year;
};

struct Passenger {
```

```
    char name[30];
    int age;
    char gender;
};

struct Ticket {
    int pnr;
    int trainNo;
    char from[20], to[20];
    int seat;
    struct Date doj;
    struct Passenger p;
};

struct Ticket tickets[MAX_SEATS];
int booked = 0;
int nextPnr = 1001;

void book(void)
{
    struct Ticket *t;
    if (booked == MAX_SEATS) {
        printf("Sorry, train is full\n");
        return;
    }
    t = &tickets[booked];
    t->pnr = nextPnr++;
    t->seat = booked + 1;
    printf("Train number: ");
    scanf("%d", &t->trainNo);
    printf("From and To: ");
    scanf("%19s %19s", t->from, t->to);
    printf("Date of journey (dd mm yyyy): ");
    scanf("%d %d %d", &t->doj.day, &t->doj.month, &t->doj.year);
    printf("Passenger name, age, gender(M/F): ");
    scanf("%29s %d %c", t->p.name, &t->p.age, &t->p.gender);
    booked++;
    printf("Ticket booked! PNR = %d, Seat = %d\n", t->pnr, t->seat);
}

void show(int pnr)
{
    int i;
    for (i = 0; i < booked; i++) {
        if (tickets[i].pnr == pnr) {
            printf("PNR %d | Train %d | %s -> %s | %02d/%02d/%d | Seat %d | %s (%d, %c)\n",
                   pnr, tickets[i].trainNo, tickets[i].from, tickets[i].to,
                   tickets[i].doj.day, tickets[i].doj.month, tickets[i].doj.year,
                   tickets[i].seat, tickets[i].p.name, tickets[i].p.age, tickets[i].p.gender);
            return;
        }
    }
    printf("PNR not found\n");
}

void cancel(int pnr)
{
    int i, j;
    for (i = 0; i < booked; i++) {
        if (tickets[i].pnr == pnr) {
            for (j = i; j < booked - 1; j++)
```

```
                tickets[j] = tickets[j + 1];
            booked--;
            printf("Ticket %d cancelled\n", pnr);
            return;
        }
    }
    printf("PNR not found\n");
}

int main(void)
{
    int ch, pnr;
    do {
        printf("\n1.Book 2.Show ticket 3.Cancel 4.Exit\nChoice: ");
        scanf("%d", &ch);
        if (ch == 1) book();
        else if (ch == 2) { printf("PNR: "); scanf("%d", &pnr); show(pnr); }
        else if (ch == 3) { printf("PNR: "); scanf("%d", &pnr); cancel(pnr); }
    } while (ch != 4);
    return 0;
}
```

96. Polynomial Addition Using Structures

Topic: Structures | Course Outcome: CO4

```
#include <stdio.h>

struct Term {
    int coeff;
    int exp;
};

int readPoly(struct Term p[])
{
    int n, i;
    printf("Number of terms: ");
    scanf("%d", &n);
    printf("Enter terms (coefficient exponent), highest exponent first:\n");
    for (i = 0; i < n; i++)
        scanf("%d %d", &p[i].coeff, &p[i].exp);
    return n;
}

void display(struct Term p[], int n)
{
    int i;
    for (i = 0; i < n; i++) {
        printf("%dx^%d", p[i].coeff, p[i].exp);
        if (i < n - 1) printf(" + ");
    }
    printf("\n");
}

int main(void)
{
    struct Term a[20], b[20], c[40];
    int n1, n2, i = 0, j = 0, k = 0;

    n1 = readPoly(a);
    n2 = readPoly(b);
```

```
    while (i < n1 && j < n2) {
        if (a[i].exp == b[j].exp) {
            c[k].coeff = a[i].coeff + b[j].coeff;
            c[k++].exp = a[i].exp;
            i++; j++;
        } else if (a[i].exp > b[j].exp) {
            c[k++] = a[i++];
        } else {
            c[k++] = b[j++];
        }
    }
    while (i < n1) c[k++] = a[i++];
    while (j < n2) c[k++] = b[j++];

    printf("First  : "); display(a, n1);
    printf("Second : "); display(b, n2);
    printf("Sum    : "); display(c, k);
    return 0;
}
```

CO5: Functions, Recursion, Pointers and File Handling

97. User-Defined Functions (All Four Types)

Topic: Functions | Course Outcome: CO5

```
#include <stdio.h>

/* 1. No arguments, no return value */
void greet(void)
{
    printf("Type 1: Hello from a function with no arguments and no return value\n");
}

/* 2. Arguments, no return value */
void printSum(int a, int b)
{
    printf("Type 2: Sum = %d\n", a + b);
}

/* 3. No arguments, return value */
int readNumber(void)
{
    int n;
    printf("Type 3: Enter a number: ");
    scanf("%d", &n);
    return n;
}

/* 4. Arguments and return value */
int multiply(int a, int b)
{
    return a * b;
}

int main(void)
{
    int x;
    greet();
    printSum(10, 20);
    x = readNumber();
    printf("Type 4: %d x 5 = %d\n", x, multiply(x, 5));
    return 0;
}
```

98. Call by Value vs Call by Reference (Swap)

Topic: Functions | Course Outcome: CO5

```
#include <stdio.h>

void swapByValue(int a, int b)
{
    int t = a; a = b; b = t;
    printf("Inside swapByValue    : a = %d, b = %d\n", a, b);
}

void swapByReference(int *a, int *b)
{
    int t = *a; *a = *b; *b = t;
    printf("Inside swapByReference: a = %d, b = %d\n", *a, *b);
}
```

```
int main(void)
{
    int x = 10, y = 20;
    printf("Before: x = %d, y = %d\n", x, y);
    swapByValue(x, y);
    printf("After call by value    : x = %d, y = %d (unchanged)\n", x, y);
    swapByReference(&x, &y);
    printf("After call by reference: x = %d, y = %d (swapped)\n", x, y);
    return 0;
}
```

99. Return Multiple Values from a Function Using Pointers

Topic: Functions | Course Outcome: CO5

```
#include <stdio.h>

void stats(int a[], int n, int *min, int *max, float *avg)
{
    int i, sum = a[0];
    *min = *max = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] < *min) *min = a[i];
        if (a[i] > *max) *max = a[i];
        sum += a[i];
    }
    *avg = (float)sum / n;
}

int main(void)
{
    int a[50], n, i, min, max;
    float avg;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    stats(a, n, &min, &max, &avg);
    printf("Min = %d, Max = %d, Average = %.2f\n", min, max, avg);
    return 0;
}
```

100. Modular Prime Number Program Using Functions

Topic: Functions | Course Outcome: CO5

```
#include <stdio.h>

int isPrime(int n)
{
    int i;
    if (n < 2) return 0;
    for (i = 2; i * i <= n; i++)
        if (n % i == 0)
            return 0;
    return 1;
}

void printPrimes(int lo, int hi)
{
```

```
    int n;
    for (n = lo; n <= hi; n++)
        if (isPrime(n))
            printf("%d ", n);
    printf("\n");
}

int main(void)
{
    int n, lo, hi;
    printf("Enter a number to test: ");
    scanf("%d", &n);
    printf("%d is %s\n", n, isPrime(n) ? "prime" : "not prime");
    printf("Enter range (low high): ");
    scanf("%d %d", &lo, &hi);
    printPrimes(lo, hi);
    return 0;
}
```

101. Pass Arrays to Functions

Topic: Functions | Course Outcome: CO5

```
#include <stdio.h>

void readArray(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
}

void printArray(const int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int sumArray(const int a[], int n)
{
    int i, sum = 0;
    for (i = 0; i < n; i++)
        sum += a[i];
    return sum;
}

void doubleElements(int a[], int n)     /* arrays are passed by address */
{
    int i;
    for (i = 0; i < n; i++)
        a[i] *= 2;
}

int main(void)
{
    int a[50], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
```

```
    readArray(a, n);
    printf("Array : "); printArray(a, n);
    printf("Sum   : %d\n", sumArray(a, n));
    doubleElements(a, n);
    printf("After doubling: "); printArray(a, n);
    return 0;
}
```

102. Factorial Using Recursion

Topic: Recursion | Course Outcome: CO5

```
#include <stdio.h>

unsigned long long factorial(int n)
{
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main(void)
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 0)
        printf("Factorial not defined for negative numbers\n");
    else
        printf("%d! = %llu\n", n, factorial(n));
    return 0;
}
```

103. Fibonacci Series Using Recursion

Topic: Recursion | Course Outcome: CO5

```
#include <stdio.h>

int fib(int n)
{
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main(void)
{
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        printf("%d ", fib(i));
    printf("\n");
    return 0;
}
```

104. GCD Using Recursion

Topic: Recursion | Course Outcome: CO5

```
#include <stdio.h>

int gcd(int a, int b)
{
    if (b == 0)
        return a;
    return gcd(b, a % b);
}

int main(void)
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("GCD of %d and %d = %d\n", a, b, gcd(a, b));
    return 0;
}
```

105. Tower of Hanoi

Topic: Recursion | Course Outcome: CO5

```
#include <stdio.h>

void hanoi(int n, char from, char to, char aux)
{
    if (n == 0)
        return;
    hanoi(n - 1, from, aux, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    hanoi(n - 1, aux, to, from);
}

int main(void)
{
    int n;
    printf("Enter number of disks: ");
    scanf("%d", &n);
    hanoi(n, 'A', 'C', 'B');
    printf("Total moves = %d\n", (1 << n) - 1);
    return 0;
}
```

106. Power of a Number Using Recursion

Topic: Recursion | Course Outcome: CO5

```
#include <stdio.h>

double power(double base, int exp)
{
    if (exp == 0)
        return 1;
    if (exp < 0)
        return 1 / power(base, -exp);
    return base * power(base, exp - 1);
}

int main(void)
```

```
{
    double b;
    int e;
    printf("Enter base and exponent: ");
    scanf("%lf %d", &b, &e);
    printf("%.2f ^ %d = %.4f\n", b, e, power(b, e));
    return 0;
}
```

107. Binary Search Using Recursion

Topic: Recursion | Course Outcome: CO5

```
#include <stdio.h>

int binarySearch(int a[], int low, int high, int key)
{
    int mid;
    if (low > high)
        return -1;
    mid = low + (high - low) / 2;
    if (a[mid] == key)
        return mid;
    if (key < a[mid])
        return binarySearch(a, low, mid - 1, key);
    return binarySearch(a, mid + 1, high, key);
}

int main(void)
{
    int a[100], n, i, key, pos;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements in ascending order: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter key: ");
    scanf("%d", &key);
    pos = binarySearch(a, 0, n - 1, key);
    if (pos >= 0)
        printf("Found at index %d\n", pos);
    else
        printf("Not found\n");
    return 0;
}
```

108. Sum of Digits and String Reversal Using Recursion

Topic: Recursion | Course Outcome: CO5

```
#include <stdio.h>
#include <string.h>

int sumDigits(int n)
{
    if (n == 0)
        return 0;
    return n % 10 + sumDigits(n / 10);
}

void reverse(char s[], int start, int end)
{
```

```
    char t;
    if (start >= end)
        return;
    t = s[start]; s[start] = s[end]; s[end] = t;
    reverse(s, start + 1, end - 1);
}

int main(void)
{
    int n;
    char str[100];
    printf("Enter a number: ");
    scanf("%d", &n);
    printf("Sum of digits = %d\n", sumDigits(n < 0 ? -n : n));
    printf("Enter a string: ");
    scanf("%99s", str);
    reverse(str, 0, (int)strlen(str) - 1);
    printf("Reversed string = %s\n", str);
    return 0;
}
```

109. Pointer Basics: Address and Dereference Operators

Topic: Pointers | Course Outcome: CO5

```
#include <stdio.h>

int main(void)
{
    int x = 25;
    int *p = &x;          /* & : address-of operator */
    int **pp = &p;        /* pointer to pointer */

    printf("Value of x            : %d\n", x);
    printf("Address of x (&x)     : %p\n", (void *)&x);
    printf("Value of p (address)  : %p\n", (void *)p);
    printf("Value at p (*p)       : %d\n", *p);    /* * : dereference operator */
    printf("Address of p (&p)     : %p\n", (void *)&p);
    printf("Value via **pp        : %d\n", **pp);

    *p = 50;
    printf("After *p = 50, x = %d\n", x);
    return 0;
}
```

110. Array Traversal Using Pointer Arithmetic

Topic: Pointers | Course Outcome: CO5

```
#include <stdio.h>

int main(void)
{
    int a[] = {10, 20, 30, 40, 50};
    int n = sizeof(a) / sizeof(a[0]);
    int *p = a, i;

    printf("Using p + i:\n");
    for (i = 0; i < n; i++)
        printf("*(p + %d) = %d  at %p\n", i, *(p + i), (void *)(p + i));

    printf("Using p++:\n");
```

```
    for (p = a; p < a + n; p++)
        printf("%d ", *p);
    printf("\n");

    printf("Elements in reverse: ");
    for (p = a + n - 1; p >= a; p--)
        printf("%d ", *p);
    printf("\n");
    return 0;
}
```

111. Dynamic Memory Allocation (malloc, calloc, realloc, free)

Topic: Pointers | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *a, *b, n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    a = (int *)malloc(n * sizeof(int));          /* uninitialised memory */
    if (a == NULL) { printf("malloc failed\n"); return 1; }
    for (i = 0; i < n; i++) a[i] = i + 1;
    printf("malloc : ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);

    b = (int *)calloc(n, sizeof(int));           /* zero-initialised memory */
    if (b == NULL) { printf("calloc failed\n"); free(a); return 1; }
    printf("\ncalloc : ");
    for (i = 0; i < n; i++) printf("%d ", b[i]);

    a = (int *)realloc(a, (n + 3) * sizeof(int));  /* grow by 3 */
    if (a == NULL) { printf("realloc failed\n"); free(b); return 1; }
    for (i = n; i < n + 3; i++) a[i] = (i + 1) * 10;
    printf("\nrealloc: ");
    for (i = 0; i < n + 3; i++) printf("%d ", a[i]);
    printf("\n");

    free(a);
    free(b);
    return 0;
}
```

112. Dynamic Two-Dimensional Array

Topic: Pointers | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int **m, r, c, i, j;
    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

    m = (int **)malloc(r * sizeof(int *));
    for (i = 0; i < r; i++)
```

```
        m[i] = (int *)malloc(c * sizeof(int));

    printf("Enter %d elements:\n", r * c);
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &m[i][j]);

    printf("Matrix:\n");
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%4d", m[i][j]);
        printf("\n");
    }

    for (i = 0; i < r; i++)
        free(m[i]);
    free(m);
    return 0;
}
```

113. Array of Pointers to Strings

Topic: Pointers | Course Outcome: CO5

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    const char *days[] = {"Monday", "Tuesday", "Wednesday", "Thursday",
                          "Friday", "Saturday", "Sunday"};
    int n = sizeof(days) / sizeof(days[0]), i, j;
    const char *temp;

    printf("Original order:\n");
    for (i = 0; i < n; i++)
        printf("%s\n", days[i]);

    /* sort by swapping pointers, not characters */
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (strcmp(days[i], days[j]) > 0) {
                temp = days[i]; days[i] = days[j]; days[j] = temp;
            }

    printf("\nAlphabetical order:\n");
    for (i = 0; i < n; i++)
        printf("%s\n", days[i]);
    return 0;
}
```

114. Menu-Driven Calculator Using Function Pointers

Topic: Pointers | Course Outcome: CO5

```
#include <stdio.h>

double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double divide(double a, double b) { return b != 0 ? a / b : 0; }
```

```
int main(void)
{
    double (*op[4])(double, double) = {add, sub, mul, divide};
    const char *name[] = {"Addition", "Subtraction", "Multiplication", "Division"};
    int ch;
    double a, b;

    do {
        printf("\n1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit\nChoice: ");
        scanf("%d", &ch);
        if (ch >= 1 && ch <= 4) {
            printf("Enter two numbers: ");
            scanf("%lf %lf", &a, &b);
            if (ch == 4 && b == 0)
                printf("Cannot divide by zero\n");
            else
                printf("%s = %.2f\n", name[ch - 1], op[ch - 1](a, b));
        } else if (ch != 5) {
            printf("Invalid choice\n");
        }
    } while (ch != 5);
    return 0;
}
```

115. Singly Linked List: Insertion, Deletion and Traversal

Topic: Pointers | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void insertBegin(int v)
{
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->data = v;
    n->next = head;
    head = n;
}

void insertEnd(int v)
{
    struct Node *n = (struct Node *)malloc(sizeof(struct Node)), *t;
    n->data = v;
    n->next = NULL;
    if (head == NULL) { head = n; return; }
    for (t = head; t->next != NULL; t = t->next)
        ;
    t->next = n;
}

void deleteValue(int v)
{
    struct Node *t = head, *prev = NULL;
    while (t != NULL && t->data != v) {
```

```
        prev = t;
        t = t->next;
    }
    if (t == NULL) { printf("%d not found\n", v); return; }
    if (prev == NULL) head = t->next;
    else prev->next = t->next;
    free(t);
    printf("%d deleted\n", v);
}

void traverse(void)
{
    struct Node *t;
    if (head == NULL) { printf("List is empty\n"); return; }
    for (t = head; t != NULL; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

int main(void)
{
    int ch, v;
    do {
        printf("\n1.Insert at beginning 2.Insert at end 3.Delete 4.Display 5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("Value: "); scanf("%d", &v); insertBegin(v); break;
            case 2: printf("Value: "); scanf("%d", &v); insertEnd(v); break;
            case 3: printf("Value to delete: "); scanf("%d", &v); deleteValue(v); break;
            case 4: traverse(); break;
        }
    } while (ch != 5);
    while (head != NULL) {            /* release memory */
        struct Node *t = head;
        head = head->next;
        free(t);
    }
    return 0;
}
```

116. Write and Read a Text File

Topic: File Handling | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    FILE *fp;
    char line[100];

    fp = fopen("sample.txt", "w");
    if (fp == NULL) { printf("Cannot open file\n"); exit(1); }
    printf("Enter text (type END on a new line to stop):\n");
    while (1) {
        fgets(line, sizeof(line), stdin);
        if (line[0] == 'E' && line[1] == 'N' && line[2] == 'D')
            break;
        fputs(line, fp);
    }
```

```
    fclose(fp);

    fp = fopen("sample.txt", "r");
    if (fp == NULL) { printf("Cannot open file\n"); exit(1); }
    printf("\nFile contents:\n");
    while (fgets(line, sizeof(line), fp) != NULL)
        printf("%s", line);
    fclose(fp);
    return 0;
}
```

117. Copy the Contents of One File to Another

Topic: File Handling | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    FILE *src, *dest;
    char srcName[50], destName[50];
    int ch;

    printf("Enter source file name     : ");
    scanf("%49s", srcName);
    printf("Enter destination file name: ");
    scanf("%49s", destName);

    src = fopen(srcName, "r");
    if (src == NULL) { printf("Cannot open %s\n", srcName); exit(1); }
    dest = fopen(destName, "w");
    if (dest == NULL) { printf("Cannot create %s\n", destName); fclose(src); exit(1); }

    while ((ch = fgetc(src)) != EOF)
        fputc(ch, dest);

    printf("File copied successfully\n");
    fclose(src);
    fclose(dest);
    return 0;
}
```

118. Count Characters, Words and Lines in a File

Topic: File Handling | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

int main(void)
{
    FILE *fp;
    char name[50];
    int ch, inWord = 0;
    long chars = 0, words = 0, lines = 0;

    printf("Enter file name: ");
    scanf("%49s", name);
    fp = fopen(name, "r");
    if (fp == NULL) { printf("Cannot open file\n"); exit(1); }
```

```
    while ((ch = fgetc(fp)) != EOF) {
        chars++;
        if (ch == '\n')
            lines++;
        if (isspace(ch))
            inWord = 0;
        else if (!inWord) {
            inWord = 1;
            words++;
        }
    }
    fclose(fp);
    printf("Characters = %ld\nWords      = %ld\nLines      = %ld\n", chars, words, lines);
    return 0;
}
```

119. Store and Retrieve Structure Records Using a Binary File

Topic: File Handling | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void)
{
    FILE *fp;
    struct Student s;
    int n, i;

    /* Store records */
    fp = fopen("students.dat", "wb");
    if (fp == NULL) { printf("Cannot open file\n"); exit(1); }
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Student %d - roll, name, marks: ", i + 1);
        scanf("%d %29s %f", &s.roll, s.name, &s.marks);
        fwrite(&s, sizeof(struct Student), 1, fp);
    }
    fclose(fp);

    /* Retrieve records */
    fp = fopen("students.dat", "rb");
    if (fp == NULL) { printf("Cannot open file\n"); exit(1); }
    printf("\n%-6s %-15s %s\n", "Roll", "Name", "Marks");
    while (fread(&s, sizeof(struct Student), 1, fp) == 1)
        printf("%-6d %-15s %.2f\n", s.roll, s.name, s.marks);
    fclose(fp);
    return 0;
}
```

120. Update a Record Using fseek and ftell (Random Access)

Topic: File Handling | Course Outcome: CO5

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void)
{
    FILE *fp;
    struct Student s;
    int key, found = 0;
    long pos;

    /* Run program 119 first so that students.dat exists */
    fp = fopen("students.dat", "rb+");
    if (fp == NULL) { printf("Cannot open students.dat\n"); exit(1); }

    printf("Enter roll number to update: ");
    scanf("%d", &key);

    while (fread(&s, sizeof(struct Student), 1, fp) == 1) {
        if (s.roll == key) {
            found = 1;
            pos = ftell(fp);                      /* position just after this record */
            printf("Current record: %s, %.2f\n", s.name, s.marks);
            printf("Enter new name and marks: ");
            scanf("%29s %f", s.name, &s.marks);
            fseek(fp, pos - (long)sizeof(struct Student), SEEK_SET);   /* step back one record
*/
            fwrite(&s, sizeof(struct Student), 1, fp);
            printf("Record updated\n");
            break;
        }
    }
    if (!found)
        printf("Record not found\n");

    /* Show the file after the update */
    rewind(fp);
    printf("\nUpdated file:\n");
    while (fread(&s, sizeof(struct Student), 1, fp) == 1)
        printf("%-6d %-15s %.2f\n", s.roll, s.name, s.marks);
    fclose(fp);
    return 0;
}
```